

```
"""
```

```
PRCSA/HCL – consolidated kernel. Every  
capability proven separately this  
session, now wired into ONE system:  
immutable claims, append-only  
relations with RETRACTS, four-stage  
write validation, semantic  
concurrency (version-checked commits),  
projection with must-keep  
protection, affected-region validation,  
and SLE with anti-gaming  
controls. This file is the single  
source of truth going forward --  
not another isolated proof.
```

```
"""
```

```
import sqlite3, json, uuid, hashlib  
from datetime import datetime,  
timezone, timedelta
```

```
def now(): return  
datetime.now(timezone.utc).isoformat()  
def uid(p): return f"  
{p}_{uuid.uuid4().hex[:8]}"
```

```
VALID_EVIDENCE =  
{"REAL", "SIMULATED", "PREDICTED", "INTERV  
ENTION_DERIVED", "INFERRED"}  
VALID_KINDS =
```

```
{"CLAIM", "HYPOTHESIS", "PREDICTION", "QUESTION", "SIMULATION", "ACTION_PROPOSAL", "INTERPRETATION", "UNCERTAINTY", "TRANSLATION", "HOLD"}
```

```
class ValidationError(Exception): pass  
class StaleTransition(Exception): pass
```

```
class PRCSAKernel:  
    def __init__(self, path):  
        self.conn =  
sqlite3.connect(path,  
isolation_level=None)  
        # WAL mode: readers no longer  
block on a writer, and vice versa --  
        # this was never configured  
before, meaning every connection so  
far has been running SQLite's  
plain default mode. Real gap,  
        # found honestly, fixed here  
rather than assumed already handled.  
        self.conn.execute("PRAGMA  
journal_mode=WAL")  
        self.conn.execute("PRAGMA  
foreign_keys=ON")  
        self.conn.executescript("""  
CREATE TABLE IF NOT EXISTS  
claims (  
            claim_id TEXT PRIMARY KEY,  
content TEXT NOT NULL, scope_json TEXT  
NOT NULL,
```

```

        evidence_class TEXT NOT
NULL, kind TEXT NOT NULL,
falsification_test TEXT
    );
    CREATE TRIGGER IF NOT EXISTS
claims_immutable_update BEFORE UPDATE
ON claims
    BEGIN SELECT RAISE(ABORT,'INV-
001: claims immutable'); END;
    CREATE TRIGGER IF NOT EXISTS
claims_immutable_delete BEFORE DELETE
ON claims
    BEGIN SELECT RAISE(ABORT,'INV-
001: claims cannot be deleted'); END;

    CREATE INDEX IF NOT EXISTS
idx_claims_content ON claims(content);

    CREATE TABLE IF NOT EXISTS
transitions (
        seq INTEGER PRIMARY KEY
AUTOINCREMENT, transition_id TEXT NOT
NULL,
        claim_id TEXT NOT NULL,
new_status TEXT NOT NULL, reason TEXT,
        based_on_version TEXT NOT
NULL, recorded_at TEXT NOT NULL
    );

    CREATE INDEX IF NOT EXISTS
idx_transitions_claim ON

```

```

transitions(claim_id);

        CREATE TABLE IF NOT EXISTS
relations (
            relation_id TEXT PRIMARY
KEY, from_ref TEXT, to_ref TEXT,
            relation_type TEXT NOT
NULL, retracts_relation_id TEXT,
recorded_at TEXT NOT NULL
        );
        CREATE TRIGGER IF NOT EXISTS
relations_immutable_update BEFORE
UPDATE ON relations
        BEGIN SELECT RAISE(ABORT,'INV-
REL: relations immutable -- use
RETRACTS'); END;
        CREATE TRIGGER IF NOT EXISTS
relations_immutable_delete BEFORE
DELETE ON relations
        BEGIN SELECT RAISE(ABORT,'INV-
REL: canonical relations cannot be
deleted -- use RETRACTS'); END;

        -- self-observation:
classifier's own confidence, linked to
a transition
        CREATE TABLE IF NOT EXISTS
self_observations (
            obs_id TEXT PRIMARY KEY,
transition_id TEXT NOT NULL,
            confidence REAL NOT NULL,

```

```

ambiguity_flags_json TEXT NOT NULL,
recorded_at TEXT NOT NULL
);

-- EVERY operator interaction
gets logged, admitted or not.
-- 'first live week is a
corpus, not a scoreboard' -- raw text +
-- parse result + final
outcome, always, regardless of success.
CREATE TABLE IF NOT EXISTS
operator_log (
    log_id TEXT PRIMARY KEY,
    task TEXT NOT NULL, model_id TEXT NOT
    NULL,
    raw_text TEXT NOT NULL,
    parsed_candidate_json TEXT, outcome
    TEXT NOT NULL,
    detail TEXT, latency_ms
    REAL, recorded_at TEXT NOT NULL
);

-- tracks last-use time per
claim, for disuse-based fading.
-- never deletes anything --
only informs the fade_stale() check.
CREATE TABLE IF NOT EXISTS
claim_activity (
    claim_id TEXT PRIMARY KEY,
    last_touched TEXT NOT NULL
);

```

```

-- REGION MARKS: dark-forest
(untraversed) vs walked-and-found-
nothing
-- (TRAVERSED_EMPTY) vs walked-
and-found-structure
(TRAVERSED_STRUCTURE).
-- Directly from Kitty's
"unknown forest vs known neighborhood"
answer --
-- a missing mark means
UNTRAVERSED (unknown), never treated as
empty.
-- Tracks how many times a
walked (ABOUT) edge actually led to
-- something USED (admitted),
not just traversed. Biases which
-- optional material gets
filled first under budget pressure --
-- never overrides hinge
(DEPENDS_ON/CONTRADICTS) protection.
CREATE TABLE IF NOT EXISTS
traversal_weight (
    relation_id TEXT PRIMARY
KEY, uses INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE IF NOT EXISTS
region_marks (
    region_key TEXT PRIMARY
KEY, status TEXT NOT NULL, note TEXT,

```

```

updated_at TEXT NOT NULL
    );

    -- EVENTS -- was missing from
the consolidated kernel entirely
    -- until this pass, despite
being one of the first invariants
    -- established. Immutable, same
as claims.
    CREATE TABLE IF NOT EXISTS
events (
        event_id TEXT PRIMARY KEY,
occurred_at TEXT NOT NULL,
        recorded_at TEXT NOT NULL,
stream_id TEXT, source TEXT,
        kind TEXT NOT NULL, payload
TEXT NOT NULL,
        causal_predecessors TEXT
NOT NULL DEFAULT '[]'
    );
    CREATE TRIGGER IF NOT EXISTS
events_immutable_update BEFORE UPDATE
ON events
        BEGIN SELECT RAISE(ABORT, 'INV-
001: events immutable'); END;
    CREATE TRIGGER IF NOT EXISTS
events_immutable_delete BEFORE DELETE
ON events
        BEGIN SELECT RAISE(ABORT, 'INV-
001: events cannot be deleted'); END;
    """)

```

```

# ----- core reconstruction --
-----
def current_version(self):
    rows =
list(self.conn.execute("SELECT
transition_id,claim_id,new_status FROM
transitions ORDER BY seq"))
    return
hashlib.sha256(json.dumps(rows,
sort_keys=True).encode()).hexdigest()
[:12]

def active_relations(self):
    all_rels = {r[0]: r for r in
self.conn.execute(
        "SELECT
relation_id,from_ref,to_ref,relation_ty
pe,retracts_relation_id FROM
relations")}
    retracted_targets = {r[4] for r
in all_rels.values() if
r[3]=="RETRACTS" and r[4]}
    return [r for rid,r in
all_rels.items() if r[3]!="RETRACTS"
and rid not in retracted_targets]

def replay(self):
    status = {}
    for seq, tid, cid, new_status,
reason, bov, rec in self.conn.execute(

```

```

        "SELECT
seq,transition_id,claim_id,new_status,r
eason,based_on_version,recorded_at FROM
transitions ORDER BY seq"):
        status[cid] = new_status
        active = []
        for cid, content, scope_json,
ec, kind, ftest in self.conn.execute(
        "SELECT
claim_id,content,scope_json,evidence_cl
ass,kind,falsification_test FROM
claims"):
            if status.get(cid) in
("SUPPORTED","PROVISIONAL","DISPUTED","
DELIBERATELY_OPEN"):

active.append({"claim_id":cid,"content"
:content,"scope":json.loads(scope_json)
,

"evidence_class":ec,"kind":kind})
        active.sort(key=lambda c:
c["claim_id"])
        rels = sorted([(r[1],r[2],r[3])
for r in self.active_relations()])
        fp =
hashlib.sha256(json.dumps({"claims":act
ive,"relations":rels},
sort_keys=True).encode()).hexdigest()
[:16]

        return {"active_claims":

```

```

active, "relations": rels,
"fingerprint": fp}

    @staticmethod
    def _as_set(v):
        """Non-list scope values (a
bare string, a number) were being
        compared as character sets,
which never crashed but produced
        semantically meaningless
results. Found via Grok's review,
        verified against the real file,
fixed properly."""
        if v is None: return set()
        if isinstance(v, (list, tuple,
set)): return set(v)
        return {v}

    @staticmethod
    def _scopes_overlap(a, b):
        if a is None or b is None:
return True
        shared = set(a) & set(b)
        if not shared: return True
        return
all(PRCSAKernel._as_set(a.get(k)) &
PRCSAKernel._as_set(b.get(k)) for k in
shared)

    # ----- four-stage validated
write, with concurrency check -----

```

```

-
    def _claims_matching_content(self,
content):
        """
        Targeted lookup: only claims
sharing this exact content string,
        with their current status. This
is what the duplicate check and
        aggregate check actually need -
- not the entire substrate. Uses
        idx_claims_content +
idx_transitions_claim, so cost stays
roughly
        constant regardless of total
database size, instead of scaling
        with total claim count the way
three full replay() calls did.
        """
        rows = self.conn.execute(
            "SELECT claim_id,
scope_json FROM claims WHERE
content=?", (content,)).fetchall()
        results = []
        for cid, scope_json in rows:
            status_row =
self.conn.execute(
                "SELECT new_status FROM
transitions WHERE claim_id=? ORDER BY
seq DESC LIMIT 1", (cid,)
            ).fetchone()
            status = status_row[0] if

```

```

status_row else None
        if status in
("SUPPORTED", "PROVISIONAL", "DISPUTED", "
DELIBERATELY_OPEN"):

results.append({"claim_id": cid,
"scope": json.loads(scope_json)})
        return results

        def commit_claim(self, content,
scope, evidence_class,
based_on_version,
                                kind="CLAIM",
falsification_test=None,
self_confidence=None,

ambiguity_flags=None,
supporting_events=None,

model_confidence=None,
structural_support=None):
        self.conn.execute("BEGIN")
        try:
            # concurrency check FIRST
            live =
self.current_version()
            if based_on_version !=
live:
                raise
StaleTransition(f"STALE_TRANSITION:
prepared against {based_on_version},

```

```

live is {live}")

        # stage 1: candidate
validation
        if not isinstance(kind,
str) or kind not in VALID_KINDS:
            raise
ValidationError(f"CANDIDATE: bad kind
{kind!r} (type={type(kind).__name__})")
            if kind == "HOLD":
                raise
ValidationError("CANDIDATE: HOLD is an
operator outcome, not a writable
claim")
            if not
isinstance(evidence_class, str) or
evidence_class not in VALID_EVIDENCE:
                raise
ValidationError(f"CANDIDATE: bad
evidence_class {evidence_class!r}
(type=
{type(evidence_class).__name__})")

        # REAL now REQUIRES
supporting_events -- adopted from
Grok's

        # review. This is a real
structural improvement, not just a
        # patch: it closes part of
the "SIM labeled REAL with no
        # trail" gap by

```

```

construction, rather than only catching
it
        # when the model happens to
sound confident about it.
        if evidence_class ==
"REAL":
            if not
supporting_events:
                raise
ValidationError(
                    "CANDIDATE:
REAL requires supporting_events -- an
unsupported "
                    "claim cannot
declare itself REAL by assertion alone"
                )
            for eid in
supporting_events:
                exists =
self.conn.execute("SELECT 1 FROM events
WHERE event_id=?", (eid,)).fetchone()
                if not exists:
                    raise
ValidationError(f"CANDIDATE: unknown
supporting event {eid!r}")

        # INV-003: an intervention
anywhere in the causal lineage means
        # this cannot be admitted
as independent REAL evidence, no
        # matter what the caller

```

```

declared. Lineage overrides self-
report.
        if evidence_class == "REAL"
and supporting_events:
            for eid in
supporting_events:
                if
self._lineage_includes_intervention(eid
):
                    evidence_class
= "INTERVENTION_DERIVED"
                    break
            if evidence_class ==
"INFERRED" and not falsification_test:
                raise
ValidationError("CANDIDATE: INFERRED
requires falsification_test")

        # Coupling enforcement
moved INTO commit_claim itself, not
just
        # the operator path --
confirmed gap: a direct caller of
        # commit_claim could
previously bypass the fluent-REAL check
        # entirely by skipping
run_operator_step.
        if model_confidence is not
None and structural_support is not
None:
            verdict, note =

```

```

self.couple_confidences(model_confidence, structural_support)
        if verdict ==
"HIGH_GEN_LOW_SUPPORT" and
evidence_class == "REAL":
            raise
ValidationError(f"COUPLING: {note}")

        for c in
self._claims_matching_content(content):
            if c["scope"] == scope:
                raise
ValidationError(f"CANDIDATE: duplicate
of {c['claim_id']}")

            cid = uid("claim")
            tid = uid("tx")
            self.conn.execute(
                "INSERT INTO claims
(claim_id,content,scope_json,evidence_c
lass,kind,falsification_test) VALUES
(?,?,?,?,?,?,?)",
                (cid, content,
json.dumps(scope), evidence_class,
kind, falsification_test))
            self.conn.execute(
                "INSERT INTO
transitions
(transition_id,claim_id,new_status,reas
on,based_on_version,recorded_at) VALUES
(?,?,?,?,?,?,?)",

```

```

        (tid, cid,
        "PROVISIONAL", "admit",
        based_on_version, now()))

        # stage 3: aggregate state
validation -- only claims sharing
        # this content could ever
trigger this check (the original
        # full pairwise scan over
ALL claims was checking the same
        # content==content
condition, just far more expensively).
        # Targeted lookup preserves
identical catching behavior.
        for other in
self._claims_matching_content(content):
            if other["claim_id"] ==
cid:
                continue
            if
self._scopes_overlap(other["scope"],
scope):
                raise
ValidationError(
                    f"AGGREGATE
STATE: laundered contradiction {cid}<->
{other['claim_id']}")

        # stage 4: persistence
validation -- direct single-row check,
        # not a full replay.

```

```

Confirms the specific row is visible
and
        # its status reads as
active, which is all this check ever
        # actually needed to know.
        verify_status =
self.conn.execute(
        "SELECT new_status FROM
transitions WHERE claim_id=? ORDER BY
seq DESC LIMIT 1", (cid,)).fetchone()
        if not verify_status or
verify_status[0] not in
("SUPPORTED","PROVISIONAL","DISPUTED","
DELIBERATELY_OPEN"):
            raise
ValidationError("PERSISTENCE: not
visible after commit")

        # self-observation
(metacognitive calibration channel)
        if self_confidence is not
None:
            self.conn.execute(
                "INSERT INTO
self_observations VALUES (?, ?, ?, ?, ?)",
                (uid("obs"), tid,
self_confidence,
json.dumps(ambiguity_flags or []),
now()))

        self.conn.execute("COMMIT")

```

```

        # Birth-touch: every claim
        gets an initial last_touched at
        # creation. Resolves the
        "NULL = immortal" ambiguity cleanly --
        # a claim that's never
        touched AGAIN after birth ages normally
        # through the existing
        cutoff check, no special-casing needed.
        self.touch(cid)
        return cid
    except Exception:

self.conn.execute("ROLLBACK")
    raise

    def add_relation(self, f, t, rtype,
retracts=None):
        if rtype != "RETRACTS" and f ==
t and f is not None:
            raise
ValidationError(f"RELATION: self-
relation {f!r} -> {f!r} not permitted")
        # Validate both endpoints
        actually exist before persisting --
        # found by testing that a
        relation could point at a nonexistent
        # claim_id and silently sit
        there as dead weight, never crashing,
        # never flagged, just inert.
        if rtype != "RETRACTS":
            for ref, label in [(f,

```

```

"from_ref"), (t, "to_ref")]:
    if ref is not None:
        exists =
self.conn.execute("SELECT 1 FROM claims
WHERE claim_id=?", (ref,)).fetchone()
        if not exists:
            raise
ValidationError(f"RELATION: {label}=
{ref!r} does not reference an existing
claim")
        rid = uid("rel")
        self.conn.execute("INSERT INTO
relations VALUES (?, ?, ?, ?, ?, ?)", (rid,
f, t, rtype, retracts, now()))
        self.conn.commit()
        return rid

# ----- projection -----
def reinforce_path(self, from_ref,
to_ref):
    """
    Called when a walk through this
    edge actually led to something
    admitted -- not merely
    traversed, USED. Directly implements
    Kitty's answer: choosing
    something changes the filter (bias),
    and can rewrite the map itself
    once a path is genuinely 'lived'
    (repeated), not on a single
    use.

```

```

"""
    row = self.conn.execute(
        "SELECT relation_id FROM
relations WHERE from_ref=? AND to_ref=?
AND relation_type='ABOUT'",
        (from_ref,
to_ref)).fetchone()
    if not row:
        return None
    rid = row[0]
    self.conn.execute(
        "INSERT OR REPLACE INTO
traversal_weight (relation_id, uses)
VALUES (?, COALESCE((SELECT uses FROM
traversal_weight WHERE relation_id=?),
0) + 1)",
        (rid, rid))
    self.conn.commit()
    uses =
self.conn.execute("SELECT uses FROM
traversal_weight WHERE relation_id=?",
(rid,)).fetchone()[0]
    return {"relation_id": rid,
"uses": uses, "lived": uses >= 3}

def project(self, seed_ids,
node_budget):
    state = self.replay()
    active_ids = {c["claim_id"] for
c in state["active_claims"]}
    rels = state["relations"]

```

```

        must = set(s for s in seed_ids
if s in active_ids)
        optional = set()
        log =
{"protected_dependencies": [],
"conflicts": []}
        for frm, to, rtype in rels:
            if frm in must or to in
must:
                other = to if frm in
must else frm
                if other not in
active_ids: continue
                if rtype ==
"DEPENDS_ON": must.add(other);
log["protected_dependencies"].append(ot
her)
                elif rtype ==
"CONTRADICTS": must.add(other);
log["conflicts"].append(other)
                elif rtype == "ABOUT":
optional.add(other)
                if len(must) > node_budget:
                    return {"result":
"INSUFFICIENT_PROJECTION", "must":
sorted(must), "log": log}
                # Bias optional fill by
traversal weight (choice-reinforcement)
--

        # only ever affects the
OPTIONAL/decoration pool, never must-

```

```

keep
    # hinges. A path used more
often (walked -> led to something
    # admitted) gets pulled in
first when budget can't fit everything.
    optional_list = list(optional -
must)

    weights = {}
    for cid in optional_list:
        row = self.conn.execute(
            "SELECT tw.uses FROM
relations r JOIN traversal_weight tw ON
tw.relation_id = r.relation_id "
            "WHERE r.to_ref=? AND
r.relation_type='ABOUT'",
(cid,)).fetchone()
        weights[cid] = row[0] if
row else 0
        optional_list.sort(key=lambda
cid: (-weights.get(cid, 0), cid))
        fill = optional_list[: max(0,
node_budget - len(must))]
        workspace = sorted(must |
set(fill))
        # NOTE: does NOT call touch()
here. A claim appearing in one
        # briefing is not evidence it's
broadly important -- treating
        # "was surfaced" as a freshness
signal is the same mistake as
        # treating repetition as

```

```

structural weight, just relocated to
    # usage instead of storage.
touch() must be called explicitly,
    # by something that actually
confirms deliberate use, not by
    # mere appearance in a packet.
    #
    # CRITICAL FIX: "workspace"
used to be the ONLY thing returned --
    # bare claim_id strings with
zero content. A real model reading
    # this would see {"workspace":
["claim_ab12"]} and have literally
    # nothing to reason about.
Found by direct inspection, confirmed
    # real. "workspace" (IDs) is
kept unchanged for internal callers
    # (package_solidity), and
"claims" now carries the actual content
    # a mouth needs: text,
evidence_class, kind, status, scope.
    active_by_id = {c["claim_id"]:
c for c in state["active_claims"]}
    claims_out = [active_by_id[cid]
for cid in workspace if cid in
active_by_id]
    relations_out = [
        {"from": frm, "to": to,
"type": rtype}
        for frm, to, rtype in rels
        if frm in workspace and to

```

```

in workspace
    ]
    return {"result": "PROJECTED",
"workspace": workspace, "claims":
claims_out,
        "relations":
relations_out, "log": log}

    # ----- affected-region
validation -----
    def affected_region_validate(self,
touched_claim_id):
        state = self.replay()
        touched = next((c for c in
state["active_claims"] if c["claim_id"]
== touched_claim_id), None)
        if not touched: return []
        problems = []
        for c in
state["active_claims"]:
            if c["claim_id"] ==
touched_claim_id: continue
            if c["content"] ==
touched["content"] and
self._scopes_overlap(c["scope"],
touched["scope"]):

problems.append(tuple(sorted([touched_c
laim_id, c["claim_id"]])))
        return sorted(problems)

```

```

# ----- SLE: structural +
capability + integrity + calibration --
-----

def sle_verdict(self, before_fp,
after_fp, before_caps, after_caps,
integrity_ok=True):
    structural_delta = after_fp !=
before_fp
    capability_delta = before_caps
!= after_caps
    if not integrity_ok: return
"INTEGRITY_VIOLATION"
    if not structural_delta and not
capability_delta: return
"NOTHING_HAPPENED"
    if structural_delta and not
capability_delta: return
"STORED_NOT_LEARNED"
    if capability_delta and not
structural_delta: return "UNTRACED_BUG"
    return "CANDIDATE_SLE"

def calibration_check(self,
transition_id, was_correct):
    row = self.conn.execute("SELECT
confidence FROM self_observations WHERE
transition_id=?",
(transition_id,)).fetchone()
    if not row: return None
    conf = row[0]
    if conf > 0.7 and not

```

```

was_correct: return "MISCALIBRATED"
    if conf < 0.4 and was_correct:
return "UNDERCONFIDENT"
    return "CALIBRATED"

# ----- SLE with a real probe
interface + anti-gaming enforcement ---
-----

    def run_sle(self, probes,
before_state, after_state,
integrity_ok=True):
        """
        probes: dict of {name:
callable(state)->bool/value}. MUST
contain
            at least 2 probes -- a single
probe is rejected outright, enforcing
            the established rule 'one
insensitive probe != no learning'.
        """
        if len(probes) < 2:
            raise ValidationError(
                "SLE: battery must
contain at least 2 probes -- a single
probe "
                "cannot distinguish
real capability change from an
insensitive test"
            )
        before_results = {name:
fn(before_state) for name, fn in

```

```

probes.items()}
        after_results = {name:
fn(after_state) for name, fn in
probes.items()}
        changed_probes = [n for n in
probes if before_results[n] !=
after_results[n]]

        before_fp =
hashlib.sha256(json.dumps(before_state,
sort_keys=True,
default=str).encode()).hexdigest()[:16]
        after_fp =
hashlib.sha256(json.dumps(after_state,
sort_keys=True,
default=str).encode()).hexdigest()[:16]
        structural_delta = before_fp !=
after_fp
        capability_delta =
len(changed_probes) > 0

        verdict =
self.sle_verdict(before_fp, after_fp,

tuple(sorted(before_results.items()))),

tuple(sorted(after_results.items()))),
integrity_ok)
        return {
            "verdict": verdict,
            "probes_run":

```

```

list(probes.keys()),
        "probes_changed":
changed_probes,
        "before_results":
before_results,
        "after_results":
after_results,
        "structural_delta":
structural_delta,
        "capability_delta":
capability_delta,
    }

    # ----- Confidence coupling:
    # model fluency vs structural support,
    # NEVER averaged. Was tested
    # standalone for a long stretch of this
    # build and repeatedly cited as
    # essential, but never actually wired
    # into the kernel until now. -----
    ----
    @staticmethod
    def
    couple_confidences(model_confidence,
    structural_support, threshold=0.6):
        model_high = model_confidence
    >= threshold
        support_high =
    structural_support >= threshold
        if model_high and support_high:
            return "ALIGNED_HIGH",

```

```

"proceed normally"
    if not model_high and not
support_high:
        return "ALIGNED_LOW",
"proceed cautiously, both sides agree
it's uncertain"
    if model_high and not
support_high:
        return
"HIGH_GEN_LOW_SUPPORT", (
            "model is fluent and
confident but the substrate doesn't
back it up -- "
            "shape of a plausible
hallucination, do not admit as REAL
without scrutiny"
        )
        return "LOW_GEN_HIGH_SUPPORT",
(
            "structure strongly
supports this but the model is hedging
-- "
            "possible projection gap,
the model may be missing supporting
context"
        )

# ----- Events: immutable,
cycle-checked at append, causal
# lineage feeds the INV-003 check
on claim admission. -----

```

```

def _would_create_cycle(self,
target_id, current_id, visited):
    if current_id == target_id:
        return True
    if current_id in visited:
        return False
    visited.add(current_id)
    row = self.conn.execute(
        "SELECT causal_predecessors
FROM events WHERE event_id=?",
(current_id,)).fetchone()
    if not row:
        return False
    for pred in json.loads(row[0]):
        if
self._would_create_cycle(target_id,
pred, visited):
        return True
    return False

def add_event(self, occurred_at,
kind, payload,
causal_predecessors=None,
stream_id=None, source=None):
    eid = uid("event")
    causal_predecessors =
causal_predecessors or []
    for pred in
causal_predecessors:
        if pred == eid:
            raise

```

```

ValidationError(f"EVENT: {eid} cannot
cite itself as a predecessor")
        if
self._would_create_cycle(eid, pred,
set()):
            raise
ValidationError(f"EVENT: adding {eid}
with predecessor {pred} would create a
causal cycle")
        self.conn.execute(
            "INSERT INTO events
(event_id,occurred_at,recorded_at,stream_id,source,kind,payload,causal_predecessors) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (eid, occurred_at, now(),
stream_id, source, kind,
json.dumps(payload),
json.dumps(causal_predecessors)))
        self.conn.commit()
        return eid

    def
_lineage_includes_intervention(self,
event_id, visited=None):
        visited = visited or set()
        if event_id in visited:
            return False
        visited.add(event_id)
        row = self.conn.execute(
            "SELECT kind,
causal_predecessors FROM events WHERE

```

```

event_id=?", (event_id,)).fetchone()
    if not row:
        return False
    kind, preds_json = row
    if kind == "INTERVENTION":
        return True
    for pred in
json.loads(preds_json):
        if
self._lineage_includes_intervention(pre
d, visited):
        return True
    return False

# ----- Disuse-based fading:
never deletes, only changes what's
# considered active. Distinct from
explicit RETIRED (a deliberate act) --
# this is ambient, triggered by
neglect rather than an event. -----
-
def touch(self, claim_id):
    """Called whenever a claim is
actually used (appears in a projection,
gets referenced by a new
relation, etc). Updates last-touched
time."""
    self.conn.execute(
        "INSERT OR REPLACE INTO
claim_activity (claim_id, last_touched)
VALUES (?,?)",

```

```

        (claim_id, now()))
    self.conn.commit()

    def fade_stale(self,
stale_after_days, min_connections=1):
        """
        Auto-retire (fade from active)
claims that are BOTH: not touched
        in stale_after_days, AND have
fewer than min_connections active
        relations. Never deletes --
writes a RETIRED transition, same
        mechanism as deliberate
retirement, just triggered by disuse
        instead of an explicit event.
The claim stays permanently on
        record; it just stops counting
as active.
        """
        cutoff =
(datetime.now(timezone.utc) -
timedelta(days=stale_after_days)).isofo
rmat()

        state = self.replay()
        active_ids = {c["claim_id"] for
c in state["active_claims"]}
        rels = state["relations"]
        conn_count = {}
        for f, t, rt in rels:
            conn_count[f] =
conn_count.get(f, 0) + 1

```

```

        conn_count[t] =
conn_count.get(t, 0) + 1

        # A claim that is the TARGET of
a DEPENDS_ON or CONTRADICTS relation
        # is a structural hinge
regardless of connection count -- it
may be
        # rarely-projected but still
load-bearing the one time it's needed.
        # Connection count alone was
too weak a proxy and would let a
        # critical-but-rarely-touched
dependency fade. Found by review,
        # not by a failing test --
fixed before it could cause one.
        hinge_ids = set()
        for f, t, rt in rels:
            if rt in ("DEPENDS_ON",
"CONTRADICTS"):
                if f: hinge_ids.add(f)
                if t: hinge_ids.add(t)

        faded = []
        for cid in active_ids:
            if cid in hinge_ids:
                continue # never fade
a claim something else structurally
depends on
        row = self.conn.execute(
            "SELECT last_touched

```

```

FROM claim_activity WHERE claim_id=?",
(cid,)).fetchone()
        last_touched = row[0] if
row else None
        if last_touched and
last_touched < cutoff and
conn_count.get(cid, 0) <
min_connections:
            tid = uid("tx")
            self.conn.execute(
                "INSERT INTO
transitions
(transition_id,claim_id,new_status,reas
on,based_on_version,recorded_at) VALUES
(?,?,?,?,?,?)",
                (tid, cid,
"RETIRED", f"auto-faded: unused
{stale_after_days}+ days,
{conn_count.get(cid,0)} connections",
self.current_version(), now()))
            faded.append(cid)
            self.conn.commit()
            return faded

    # ----- Package solidity: one
computed number, how much of THIS
    # projection is settled vs thin,
before it goes to the operator -----
--
    def package_solidity(self,
projection_result):

```

```

        if projection_result["result"]
        != "PROJECTED":
            return None
            workspace_ids =
set(projection_result["workspace"])
            state = self.replay()
            by_id = {c["claim_id"]: c for c
in state["active_claims"]}
            if not workspace_ids:
                return {"solidity": None,
"settled": 0, "disputed": 0, "total":
0}

            settled = disputed = 0
            for cid in workspace_ids:
                status_row =
self.conn.execute(
                    "SELECT new_status FROM
transitions WHERE claim_id=? ORDER BY
seq DESC LIMIT 1", (cid,)).fetchone()
                status = status_row[0] if
status_row else "PROVISIONAL"
                if status == "DISPUTED":
                    disputed += 1
                elif status in
("SUPPORTED",):
                    settled += 1
            total = len(workspace_ids)
            solidity = round((settled -
disputed) / total, 2) if total else
None
            return {"solidity": solidity,

```

```

"settled": settled, "disputed":
disputed, "total": total}

    # ----- Deliberate hold-open
vs thin evidence: friction that's
    # kept OPEN ON PURPOSE (structure
demands it stay unresolved) is
    # different from a HOLD that's just
"not enough support yet." -----
    REGION_STATUSES = {"UNTRAVERSED",
"TRAVERSED_EMPTY",
"TRAVERSED_STRUCTURE"}

    def mark_region(self, region_key,
status, note=None):
        if status not in
self.REGION_STATUSES:
            raise
ValidationError(f"REGION: bad status
{status!r}")
        self.conn.execute(
            "INSERT OR REPLACE INTO
region_marks (region_key, status, note,
updated_at) VALUES (?, ?, ?, ?)",
            (region_key, status, note,
now()))
        self.conn.commit()
        return status

    def region_status(self,
region_key):

```

```

        row = self.conn.execute(
            "SELECT status, note FROM
region_marks WHERE region_key=?",
(region_key,)).fetchone()
        if not row:
            # No mark = genuinely
unknown, not empty. This is the load-
bearing
            # default: absence of a
record must never be read as "nothing
here."
            return {"status":
"UNTRAVERSED", "note": None,
"implicit": True}
            return {"status": row[0],
"note": row[1], "implicit": False}

    def mark_deliberately_open(self,
claim_id, rationale):
        """Distinct from ordinary
DISPUTED: this claim's contradiction is
being kept live on purpose, not
just unresolved by default."""
        exists =
self.conn.execute("SELECT 1 FROM claims
WHERE claim_id=?",
(claim_id,)).fetchone()
        if not exists:
            raise
ValidationError(f"TRANSITION: claim_id
{claim_id!r} does not exist")

```

```

        tid = uid("tx")
        self.conn.execute(
            "INSERT INTO transitions
(transition_id,claim_id,new_status,reason,based_on_version,recorded_at) VALUES
(?,?,?,?,?,?)",
            (tid, claim_id,
            "DELIBERATELY_OPEN", rationale,
            self.current_version(), now()))
        self.conn.commit()
        return tid

print("Operator boundary wired:
ModelRequest/ModelResponse/ModelAdapter
/run_operator_step ready.")

print("Kernel module loaded.")

# ----- Operator boundary: the
actual plug for real model APIs -----
---
from dataclasses import dataclass,
field
from abc import ABC, abstractmethod
import time as _time

@dataclass
class ModelRequest:
    projection: dict          # the
workspace from kernel.project()
    task: str

```

```

        max_tokens: int = 1024

@dataclass
class ModelResponse:
    raw_text: str
    model_id: str
    latency_ms: float
    token_usage: dict =
field(default_factory=dict)
    # structured candidate, if the
operator can produce one directly
    candidate: dict = None    #
{"content":..., "scope":...,
"evidence_class":..., "kind":...}

class ModelAdapter(ABC):
    """Real providers (Anthropic,
OpenAI, xAI, etc.) subclass this. The
kernel never imports a provider SDK
directly -- only this interface."""
    name = "abstract"
    @abstractmethod
    def generate(self, request:
ModelRequest) -> ModelResponse:
        raise NotImplementedError

class MockOperator(ModelAdapter):
    """Deterministic stand-in for
pipeline testing ONLY. Per project
rule:
    never cited as evidence about real

```

```

model behavior or capability."""
    name = "mock"
    def __init__(self, label="mock",
fixed_response=None):
        self.label = label
        self.fixed_response =
fixed_response
    def generate(self, request:
ModelRequest) -> ModelResponse:
        start = _time.time()
        candidate = self.fixed_response
or {
            "content": f"[{self.label}]"
response to: {request.task}",
            "scope": {},
"evidence_class": "INFERRED", "kind":
"INTERPRETATION",
        }
        return ModelResponse(

raw_text=candidate["content"],
model_id=self.label,
            latency_ms=(_time.time()-
start)*1000, token_usage={"input": 0,
"output": 0},
            candidate=candidate,
        )

def _log_operator_interaction(kernel,
task, response, parsed, outcome,

```

```

detail=None):
    kernel.conn.execute(
        "INSERT INTO operator_log
(log_id,task,model_id,raw_text,parsed_c
andidate_json,outcome,detail,latency_ms
,recorded_at) VALUES
(?,?,?,?,?,?,?,?,?,?)",
        (uid("log"), task,
response.model_id, response.raw_text,
        json.dumps(parsed) if parsed
is not None else None,
        outcome, detail,
response.latency_ms, now()),
    )
    kernel.conn.commit()

```

```

def run_operator_step(kernel:
"PRCSAKernel", adapter: ModelAdapter,
task: str,
                                seed_ids: list,
node_budget: int,
self_confidence=None):
    """
    The full closed loop: project ->
operator -> candidate -> validated
    commit. Every interaction is logged
regardless of outcome -- 'first
    live week is a corpus, not a
scoreboard.' HOLD is a real,
intentional

```

```
outcome distinct from parse
failure: the operator can explicitly
say
    "not enough to commit" rather than
being forced into silence or a
    malformed response looking the same
as a deliberate withholding.
"""
    projection =
kernel.project(seed_ids, node_budget)
    if projection["result"] ==
"INSUFFICIENT_PROJECTION":
        # No operator call happens at
all -- no cost, no log entry needed
        # since the adapter was never
invoked.
        return {"stage": "PROJECTION",
"result": "INSUFFICIENT_PROJECTION",
"detail": projection}

    request =
ModelRequest(projection=projection,
task=task)
    response =
adapter.generate(request)

    if not response.candidate:

_log_operator_interaction(kernel, task,
response, None,
"NO_STRUCTURED_CANDIDATE")
```

```

        return {"stage": "OPERATOR",
                "result": "NO_STRUCTURED_CANDIDATE",
                "raw_text": response.raw_text,
                    "model_id":
response.model_id, "latency_ms":
response.latency_ms}

        c = response.candidate

        # Confidence coupling: estimate
        structural support from the projection
        # itself (how much of the workspace
        it drew on was settled vs thin),
        # pair it against the model's self-
        reported confidence if given, and
        # flag the dangerous
        HIGH_GEN_LOW_SUPPORT case for the
        caller to see
        # -- without ever averaging the two
        into one number.
        coupling_note = None
        if c.get("kind") not in (None,
                                "HOLD") and "model_confidence" in c:
            solidity =
kernel.package_solidity(projection)
            structural_support =
solidity["solidity"] if solidity and
solidity["solidity"] is not None else
0.5

            verdict, note =
kernel.couple_confidences(c["model_conf

```

```

idence"], structural_support)
    coupling_note = {"verdict":
verdict, "note": note}
    # The verdict must actually
gate, not just annotate -- found by
    # testing that a fluent-but-
unsupported REAL claim was sailing
    # through with only a note
attached, which defeats the entire
    # point of computing the
verdict in the first place.
    if verdict ==
"HIGH_GEN_LOW_SUPPORT" and
c.get("evidence_class") == "REAL":

_log_operator_interaction(kernel, task,
response, c, "REJECTED",

f"coupling gate: {note}")
    return {"stage":
"COUPLING_GATE", "result": "REJECTED",
        "reason": note,
"coupling": coupling_note,
        "model_id":
response.model_id, "latency_ms":
response.latency_ms}

    if c.get("kind") == "HOLD":
        # Intentional, legitimate
outcome -- the operator looked at the
        # workspace and deliberately

```

```
declined to propose a claim. This
    # is NOT a failure. It's the
model correctly refusing premature
    # commitment, exactly what the
whole project wants to see happen
    # when support is thin.
```

```
_log_operator_interaction(kernel, task,
response, c, "HOLD", c.get("content"))
    return {"stage": "OPERATOR",
"result": "HOLD", "reason":
c.get("content"),
            "model_id":
response.model_id, "latency_ms":
response.latency_ms}
```

```
    if "kind" not in c or
"evidence_class" not in c:
```

```
_log_operator_interaction(kernel, task,
response, c, "INCOMPLETE_CANDIDATE")
    return {"stage": "OPERATOR",
"result": "INCOMPLETE_CANDIDATE",
            "detail": "candidate
missing required field(s): kind and/or
evidence_class",
            "model_id":
response.model_id, "latency_ms":
response.latency_ms}
    try:
        cid = kernel.commit_claim(
```

```

        c["content"],
c.get("scope", {}),
c["evidence_class"],
        kernel.current_version(),
kind=c["kind"],

falsification_test=c.get("falsification
_test"),

self_confidence=self_confidence,

supporting_events=c.get("supporting_eve
nts"),

model_confidence=c.get("model_confidenc
e"),
        structural_support=
(kernel.package_solidity(projection) or
{}).get("solidity"),
    )

_log_operator_interaction(kernel, task,
response, c, "ADMITTED", cid)
    return {"stage": "COMMITTED",
"result": "ADMITTED", "claim_id": cid,
        "model_id":
response.model_id, "latency_ms":
response.latency_ms,
        "token_usage":
response.token_usage, "coupling":
coupling_note}

```

```
        except (ValidationError,
                StaleTransition) as e:

            _log_operator_interaction(kernel, task,
                                     response, c, "REJECTED", str(e))
            return {"stage": "GATE",
                    "result": "REJECTED", "reason": str(e),
                    "model_id":
                    response.model_id, "latency_ms":
                    response.latency_ms}
```